Answer Chapter 2 — Full Solutions

Course: Web Development **Student:** Abdelrahman Mohamed

A. Multiple Choice Questions

1: (a) The client	**2:** (d) each request is handled independently
3: (c) The folder's default document (index.html)	**4:** (d) 443
5: (b) The authoritative name server	**6:** (a) Resolvers cache answers until TTL expires
7: (c) term=2	**8:** (d) %20
9: (b) /courses/index.html	**10:** (a) GET /news.html HTTP/1.1
11: (c) 304 Not Modified	**12:** (d) The server's program and error logs

B. True/False Questions

1: False (HTTP requests are client-initiated)	**2:** True (IP locates device, port identifies service)
3: False (Domain names read right-to-left)	**4:** False (URL fragments are processed client-side)
5: False (GET can carry a payload, though uncommon)	**6:** False (POST requires TLS/HTTPS for encryption)
7: True (4xx denotes client errors)	**8:** False (Proves identity & encryption, not intent)

C. Short Answer Questions

1. Client-Server Architecture & Statelessness: Clients initiate requests and servers process them. Statelessness means the server does not retain user state between requests, ensuring scalability.

2. DNS Resolution Steps: Browser checks local cache → OS resolver → Root DNS → TLD server → Authoritative DNS server returns the target IP to cache and connect.

3. Absolute vs. Relative vs. Root-Relative Links: Absolute specifies full URL (external sites); relative points within the current path; root-relative (starts with `/`) targets path from root domain.

4. GET vs. POST Comparison: GET retrieves data via URL parameters; POST sends payload securely inside the request body for modifications or forms.

5. Guarantees of HTTPS: Provides confidentiality, integrity, and server authentication. Padlock does not guarantee website owner honesty or safe content.

D. Essay Questions

1. Login Request Lifecycle: Browser resolves portal domain via DNS → TLS Handshake establishes HTTPS → Client submits POST form → Server validates credentials → Responds with session cookie/token.

2. Why Brochure Sites Need HTTPS: Guarantees content integrity against MITM tampering, ensures privacy against ISP eavesdropping, and avoids browser security warnings.

3. Using Status Codes, Headers & DevTools: Developers monitor network tab status (404/500), inspect HTTP headers for authorization or CORS, and use request logs to identify server crashes.

E. Practical Programming Questions (1 to 3)

1. status-codes.html:

```
<!DOCTYPE html> <html lang="en"> <head><title>Status Codes</title></head> <body> <h1>HTTP Status Codes</h1> <h2>2xx Success</h2> <p>200 OK: Standard successful request response. 201 Created: Resource created successfully.</p> <h2>4xx Client Error</h2> <p>404 Not Found: Resource missing. 403 Forbidden: Server refuses authorized access.</p> </body> </html>
```

2. links.html:

```
<!DOCTYPE html> <html lang="en"> <head><title>Links Test</title></head> <body> <a href="https://example.com">Absolute Link</a> <a href="subfolder/page.html">Relative Subfolder</a> <a href="../index.html">Parent Directory (../)</a> <a href="/docs/about.html">Root Relative</a> <a href="search.html?user=123">Query String</a> <a href="#footer">Fragment Link</a> </body> </html>
```

3. http-messages.html:

```
<!DOCTYPE html> <html lang="en"> <head> <title>HTTP Messages</title> <link rel="stylesheet" href="style.css"> </head> <body> <pre> <span class="request-line">GET /index.html HTTP/1.1</span> <span class="headers">Host: example.com User-Agent: Mozilla/5.0</span> <span class="body">{"request": "data"} </span> </pre> </body> </html>
```

E. Practical Programming Questions (4 to 6)

4. location-inspector.html:

```
<!DOCTYPE html> <html lang="en"> <head><title>Location Properties</title></head> <body> <script> const props = ['href', 'protocol', 'host', 'pathname', 'search', 'hash']; props.forEach(p => { document.write(`<p> <strong>location.${p}:</strong> ${location[p]}</p>`); }); </script> </body> </html>
```

5. Multi-page Structure (index.html, about/index.html, docs/http.html, docs/dns.html):

```
<!-- Navigation component shared across site using root-relative link --> <link rel="stylesheet" href="/css/site.css"> <nav> <a href="/">Home</a> | <a href="/about/">About</a> | <a href="/docs/http.html">HTTP Docs</a> | <a href="/docs/dns.html">DNS Docs</a> </nav>
```

6. Request Builder (Challenge Exercise):

```
<!DOCTYPE html> <html lang="en"> <head><title>Request Builder</title></head> <body> <h2>HTTP Request Builder</h2> <button onclick="generate('GET', '/index.html')">GET /index.html</button> <button onclick="generate('POST', '/submit')">POST /submit</button> <button onclick="generate('GET', '/api/data')">GET /api/data</button> <pre id="output" style="background:#f1f5f9; padding:10px; margin-top:10px;"></pre> <script> function generate(method, path) { let req = `${method} ${path} HTTP/1.1\nHost: ${location.host || 'localhost'}\n`; if (method === 'POST') { req += `Content-Type: application/json\nContent-Length: 27\n\n{"status": "success", "id": 1}`; } document.getElementById('output').textContent = req; } </script> </body> </html>
```